

CrisisReady

Trening reakcji kryzysowych w Mixed Reality oparty na mikroprogresji

Sprawozdanie z projektu zaliczeniowego z Technologii Kognitywnych

Zespół:

Wiktoria Bartek

Jakub Cieniuch

Magdalena Grzesiak

Oliwia Frueauff

Adriana Jankowiak

09.06.2026

Spis treści

1. STRESZCZENIE	3
2. WPROWADZENIE: PROBLEM, KONTEKST I CEL PROJEKTU	4
3. UZASADNIENIE KOGNITYWNE I ODNIESIENIE DO ARTYKUŁU WZORCOWEGO.....	5
4. PYTANIA BADAWCZE, HIPOTEZY I KRYTERIA SUKCESU	7
5. KONCEPCJA SYSTEMU I SCENARIUSZ UŻYTKOWNIKA.....	8
6. MIKROPROGRESJA I MAPA KROKÓW	9
7. PĘTLA POZNAWCZA SYSTEMU: SENSE – INTERPRET – DECIDE – ACT – FEEDBACK	11
8. UX/UI I PROJEKTOWANIE INTERAKCJI KRYZYSOWEJ.....	12
9. ARCHITEKTURA TECHNICZNA	13
10. WYKORZYSTANE TECHNOLOGIE, NARZĘDZIA I ZASOBY	16
11. FUNKCJONALNOŚĆ PROGRAMU	17
12. LOGOWANIE DANYCH I PRZYGOTOWANIE WALIDACJI	18
13. PROCES WYTWÓRCZY, HARMONOGRAM I METODYKA PRACY	19
14. PODZIAŁ OBOWIĄZKÓW W ZESPOLE.....	22
15. TESTOWANIE TECHNICZNE	23
16. PROPOZYCJA BADANIA UŻYTKOWNIKÓW I PROCEDURA WALIDACJI	24
17. METRYKI EFEKTYWNOŚCI, UX I OBCIĄŻENIA POZNAWCZEGO	25
18. WYZWANIA ETYCZNE, PRYWATNOŚĆ I BEZPIECZEŃSTWO	29
19. WYKORZYSTANIE AI	30
20. SCREENY, DIAGRAMY I DOKUMENTACJA DZIAŁANIA	31
21. NIEPOWODZENIA, OGRANICZENIA I WNIOSKI Z BŁĘDÓW	33
22. UPOWSZECHNIENIE PROJEKTU	36
23. DALSZY ROZWÓJ SYSTEMU	37
24. WNIOSKI KOŃCOWE.....	39
BIBLIOGRAFIA	40
SPIS TABEL	41

1. Streszczenie

CrisisReady jest prototypem aplikacji Mixed Reality/Virtual Reality przeznaczonej na gogle Meta Quest 3/3S. System realizuje trening reakcji w sytuacjach kryzysowych poprzez jedną ciągłą sesję fabularno-treningową, składającą się z trzech modułów:

- wyjścia z poślizgu samochodem (KIEROWNICA),
- komunikacji alarmowej z numerem 112 (MOWA) oraz
- zachowania ciszy podczas ukrycia przed napastnikiem (CISZA).

Projekt nie został zaprojektowany wyłącznie jako gra lub demonstracja VR, lecz jako technologia kognitywna – system zbiera dane o zachowaniu użytkownika, interpretuje je, podejmuje decyzje o przebiegu scenariusza, uruchamia konsekwencje działania i dostarcza feedback wspierający uczenie.

Główną ramą teoretyczną projektu jest **mikroprogresja**, rozumiana jako rozbitcie złożonej umiejętności na krótkie, mierzalne i stopniowo integrowane kroki. Inspiracją do stworzenia projektu był artykuł *MR.Drum*, w którym zaproponowano wykorzystanie Mixed Reality do nauki złożonych koordynacyjnie umiejętności przez oddzielenie warstwy poznawczej, motorycznej i integracyjnej (Wang et al., 2025). W **CrisisReady** podobna logika została przeniesiona z nauki gry na perkusji na trening zachowania kryzysowego: użytkownik najpierw uczy się prostych reakcji, a następnie wykonuje je w zintegrowanym, stresującym scenariuszu.

Projekt osiągnął postać grywalnego prototypu działającego na fizycznym sprzęcie Meta Quest 3/3S. Zaimplementowano sceny MR/VR, menu, tryb treningowy, śledzenie dłoni, dialog głosowy, detekcję ciszy, detektory mikrokroków, system aktów, podstawowe logowanie metryk do pliku JSONL oraz strukturę kwestionariuszy badawczych.

Prototyp umożliwia przejście pełnej sekwencji:

jazda – poślizg – wypadek – ukrycie – telefon 112 – cisza – wygrana lub przegrana

2. Wprowadzenie: problem, kontekst i cel projektu

Szkolenia dotyczące zachowania w sytuacjach kryzysowych są zwykle prowadzone w formie instruktaży, prezentacji lub ćwiczeń symulacyjnych. Takie formy mogą przekazywać wiedzę deklaratywną, lecz często nie uczą automatycznych odruchów potrzebnych w sytuacji presji czasu, wysokiego pobudzenia emocjonalnego i niepewności. Użytkownik może wiedzieć, co powinien zrobić, ale niekoniecznie potrafi wykonać to działanie w stresie, kiedy rośnie obciążenie poznawcze, a uwaga koncentruje się na najbardziej wyrazistych bodźcach.

Mixed Reality i Virtual Reality pozwalają częściowo rozwiązać ten problem, ponieważ łączą bezpieczeństwo treningu z możliwością wywołania poczucia obecności, presji i konieczności działania. W *CrisisReady* realne otoczenie użytkownika, widoczne przez passthrough, zostaje włączone do scenariusza, a następnie zastąpione pełną sceną VR. Użytkownik nie tylko czyta instrukcje, ale wykonuje działanie ciałem, głosem i kontrolą zachowania.

Celem projektu było zaprojektowanie i zaimplementowanie prototypu systemu MR/VR, który wspiera trening reakcji kryzysowych przez mikroprogresję. System miał uczyć krótkich reakcji, mierzyć ich wykonanie oraz dostarczać feedback w taki sposób, aby nie zwiększać nadmiernie frustracji i obciążenia poznawczego. Projekt miał także przygotować podstawę do dalszej walidacji naukowej: logowania danych, badań UX, pomiaru obciążenia poznawczego i analizy efektywności uczenia.

Tabela 1. Problem projektowy i odpowiedź systemu *CrisisReady*

Problem projektowy	Odpowiedź systemu <i>CrisisReady</i>
Trudności w treningu reakcji pod presją	Immersyjny scenariusz kryzysowy z konsekwencjami błędów
Nadmierna złożoność pełnych procedur	Rozbicie na mikrokroki i osobny tryb treningowy
Brak obiektywnego pomiaru postępu	Logowanie czasu, błędów, prób i ukończenia etapów
Ryzyko przeciążenia poznawczego	Krótki, kontekstowy feedback i ograniczona liczba komunikatów
Trudność transferu wiedzy do działania	Uczenie przez ruch, głos i kontrolę zachowania w scenariuszu

Źródło: opracowanie własne

3. Uzasadnienie kognitywne i odniesienie do artykułu wzorcowego

Projekt został oparty na założeniu, że technologia kognitywna nie jest jedynie narzędziem prezentowania treści, lecz systemem wpływającym na percepcję, uwagę, decyzje, działanie, emocje i poczucie sprawczości użytkownika. W tym sensie **CrisisReady** nie jest tylko aplikacją szkoleniową w VR. Jest systemem zamkniętej pętli, w którym dane sensoryczne i behawioralne użytkownika wpływają na przebieg doświadczenia.

Artykuł *MR.Drum: Designing Mixed Reality Interfaces to Support Structured Learning Micro-Progression in Drumming* stanowił punkt odniesienia dla sposobu myślenia o projekcie (Wang et al., 2025). Autorzy zajmowali się problemem samodzielnej nauki gry na perkusji – umiejętności wymagającej jednoczesnego rozumienia rytmu, kontroli sekwencji dźwięków oraz koordynacji ruchów rąk i nóg. Tradycyjne materiały instruktażowe, takie jak filmy wideo, mogą pokazywać poprawne wykonanie, ale nie zawsze pomagają początkującemu użytkownikowi przełożyć obserwację na własny ruch i poprawne wykonanie zadania.

W *MR.Drum* zastosowano Mixed Reality, aby połączyć realny zestaw perkusyjny z wirtualnymi wskazówkami widocznymi z perspektywy pierwszej osoby. System wykorzystywał wirtualne kończyny, regulację tempa oraz automatyczne wykrywanie błędów, dzięki czemu użytkownik mógł ćwiczyć kolejne elementy złożonej umiejętności w bardziej uporządkowany sposób. Ważne było także to, że autorzy nie zaprojektowali systemu przypadkowo, lecz poprzedzili go badaniami formatywnymi dotyczącymi metod pracy instruktorów oraz trudności osób uczących się samodzielnie. Na tej podstawie opracowali strukturę mikroprogresji, w której nauka została podzielona na mniejsze, możliwe do przećwiczenia etapy.

Do **CrisisReady** przeniesiono nie temat artykułu, lecz jego model projektowy. W *MR.Drum* mikroprogresja służyła nauce rytmu, sekwencji i koordynacji kończyn, natomiast w **CrisisReady** ta sama logika została zastosowana do treningu zachowań kryzysowych. Zamiast uderzeń w perkusję użytkownik ćwiczy reakcję na poślizg, komunikację z numerem alarmowym 112 oraz zachowanie ciszy w sytuacji zagrożenia. W obu projektach złożone działanie zostaje rozbite na krótsze mikrokroki, które można osobno zrozumieć, wykonać, ocenić i połączyć w pełną sekwencję.

Różnica między projektami dotyczy przede wszystkim rodzaju trenowanej umiejętności i charakteru obciążenia użytkownika. W *MR.Drum* głównym wyzwaniem jest koordynacja

ruchowa i temporalna, czyli wykonanie ruchu w odpowiednim rytmie i czasie. W *CrisisReady* trudność ma charakter bardziej sytuacyjny i afektywny: użytkownik musi działać pod presją, kontrolować ruch, wypowiedź i zachowanie mimo narastającego napięcia. Dlatego *MR.Drum* potraktowano jako wzór strukturalny, pokazujący, jak projektować system MR oparty na mikroprogresji, feedbacku i pomiarze skuteczności, a nie jako wzór tematyczny.

Drugim ważnym pojęciem jest scaffolding, czyli tymczasowe wsparcie udzielane użytkownikowi w trakcie nabywania umiejętności (Wood et al., 1976). W prototypie scaffolding pojawia się w trybie TRENING, w prostych instrukcjach oraz w podziale zadania na rozpoznawalne mikrokroki. Docelowo wsparcie powinno być stopniowo ograniczane, aby sprawdzić, czy użytkownik potrafi wykonać zadanie samodzielnie.

Trzecia rama teoretyczna dotyczy obciążenia poznawczego. W sytuacji kryzysowej użytkownik musi jednocześnie przetwarzać bodźce, podejmować decyzje, kontrolować działanie i regulować emocje. Zgodnie z teorią obciążenia poznawczego nadmiar informacji może utrudniać uczenie i wykonanie zadania (Sweller, 1988). Dlatego interfejs *CrisisReady* został pomyślany jako kryzysowy, a nie dekoracyjny: ma ograniczać liczbę komunikatów, skracać instrukcje i dostarczać feedback dokładnie wtedy, gdy jest potrzebny.

4. Pytania badawcze, hipotezy i kryteria sukcesu

Projekt był przede wszystkim prototypem implementacyjnym, ale od początku zakładał możliwość walidacji empirycznej. Pytania badawcze zostały sformułowane tak, aby można było je sprawdzić w przyszłym badaniu z użytkownikami, wykorzystując logi systemowe, kwestionariusze oraz obserwację zachowania.

Tabela 2. Pytania badawcze i proponowane wskaźniki walidacji

Numer	Pytanie badawcze	Proponowany wskaźnik
PB1	Czy mikroprogresja w MR/VR skraca czas reakcji w scenariuszach kryzysowych?	czas do pierwszej poprawnej reakcji, czas ukończenia mikrokroku
PB2	Czy liczba błędów krytycznych spada po treningu?	liczba błędnych reakcji, liczba restartów, liczba podpowiedzi
PB3	Które mikrokroki są najtrudniejsze dla użytkowników?	najczęściej powtarzane lub nieukończone mikrokroki
PB4	Czy użytkownik po treningu potrzebuje mniej podpowiedzi?	liczba komunikatów pomocniczych na sesje
PB5	Czy nabyta umiejętność przenosi się na zmieniony wariant scenariusza?	wynik w teście transferu
PB6	Czy system poprawia wykonanie bez nadmiernego wzrostu obciążenia poznawczego?	Wynik NASA-TLX, czas, błędy i subiektywna frustracja
PB7	Czy interfejs jest zrozumiały pod presją czasu?	SUS, obserwacje i wywiad po sesji

Źródło: opracowanie własne

Hipoteza główna: trening z wykorzystaniem **CrisisReady** poprawi skuteczność wykonania mikrokroków kryzysowych w porównaniu z wykonaniem przed treningiem, przy utrzymaniu akceptowalnego poziomu obciążenia poznawczego i dyskomfortu.

Hipotezy szczegółowe zakładają, że po treningu użytkownicy będą szybciej reagować, popełniać mniej błędów, rzadziej korzystać z podpowiedzi oraz lepiej oceniać zrozumiałość zadania.

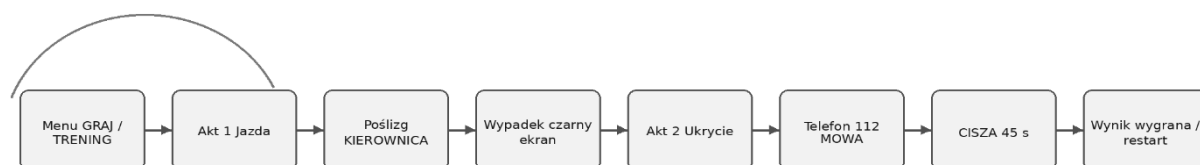
Tabela 3. Kryteria sukcesu prototypu **CrisisReady**

Kryterium sukcesu	Minimalny oczekiwany rezultat
Działanie techniczne	Prototyp uruchamia się na Quest 3/3S i pozwala przejść pełną sesję
Mikroprogresja	Każdy moduł posiada rozpoznawalne mikrokroki i kryteria ich wykonania
Closed-loop	System reaguje na dane użytkownika, a nie tylko odtwarza scenę
Walidacja	System zapisuje dane umożliwiające analizę po sesji
UX	Użytkownik rozumie cel zadania i potrafi uruchomić scenariusz

Źródło: opracowanie własne

5. Koncepcja systemu i scenariusz użytkownika

CrisisReady zostało zaprojektowane jako ciągła sekwencja zdarzeń, a nie trzy niepowiązane minigry. Użytkownik rozpoczyna w menu, wybiera tryb gry lub treningu, następnie wchodzi w scenariusz prowadzący od jazdy samochodem przez wypadek do ukrycia się i kontaktu z numerem alarmowym. Taka struktura wzmacnia poczucie ciągłości i pozwala integrować trzy typy umiejętności: motoryczną, komunikacyjną i samoregulacyjną.



Rysunek 1. Schemat przebiegu scenariusza użytkownika

Scenariusz można streścić jako sekwencję: jazda samochodem - poślizg - wypadek - przebudzenie w sypialni - ukrycie pod łóżkiem - telefon 112 - zachowanie ciszy - przyjazd policji albo przegrana. W warstwie narracyjnej użytkownik przechodzi przez sytuację narastającego zagrożenia, natomiast w warstwie treningowej wykonuje trzy oddzielne zadania: kontrolę ruchu, komunikację i zahamowanie reakcji.

Tabela 4. Fazy scenariusza, tryb środowiska i funkcje poznawcze

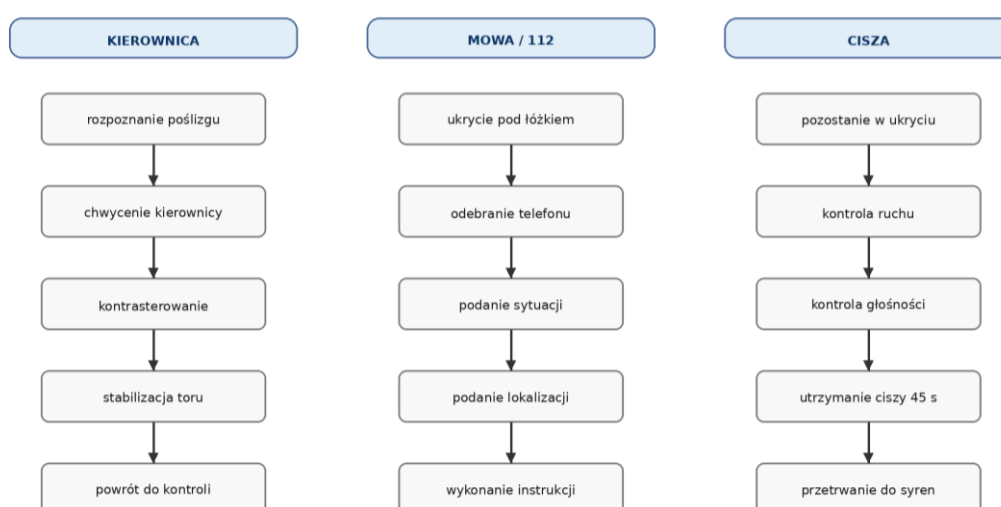
Faza scenariusza	Tryb środowiska	Zadanie użytkownika	Funkcja poznawcza
Menu i trening	MR/VR	Wybór GRAJ lub TRENING	Orientacja w interfejsie, rozumienie celu
Jazda	Passthrough + elementy VR	Chwyt kierownicy i reakcja na poślizg	Uwaga wzrokowa, koordynacja, decyzja motoryczna
Wypadek	VR /czarny ekran	Przejście narracyjne do kolejnego aktu	Przetwarzanie konsekwencji zdarzenia
Ukrycie	VR	Schylenie głowy i ukrycie się pod łóżkiem	Kontrola pozycji ciała, adaptacja do sytuacji
Telefon 112	VR + audio	Wypowiedzenie formułek alarmowych	Język, pamięć robocza, kontrola emocji
Cisza	VR + mikrofon	Utrzymanie ciszy przez 45 sekund	Samoregulacja, hamowanie reakcji, odporność na stres
Wynik	Ekran końcowy	Restart lub zakończenie	Feedback, uczenie się na błędzie

Źródło: opracowanie własne

6. Mikroprogresja i mapa kroków

Mikroprogresja oznacza, że użytkownik nie jest od razu wrzucany w pełną, wieloetapową procedurę. Złożone działanie zostaje rozbite na mniejsze jednostki, które można osobno ćwiczyć, mierzyć i wzmacniać feedbackiem. W **CrisisReady** każdy z trzech modułów odpowiada innemu typowi umiejętności:

- KIEROWNICA trenuje reakcję motoryczną,
- MOWA trenuje uporządkowaną komunikację,
- CISZA trenuje samokontrolę.



Rysunek 2. Mapa mikroprogresji w modułach

Tabela 5. Mikro kroki, dane wejściowe i przykładowe kryteria zaliczenia

Moduł	Mikrokrok	Dane wejściowe	Przykładowe zaliczenie kryterium
KIEROWNICA	Chwycenie kierownicy	Pozycja dłoni	Dłoń w obszarze kierownicy przez minimalny czas
KIEROWNICA	Kontrasterowanie	Obrót kierownicy/ gest	Skręt w odpowiednim kierunku w oknie czasowym
MOWA	Wypowiedź 112	Mikrofon	Wykrycie słów kluczowych lub formuлки
CISZA	Utrzymanie ciszy	Mikrofon/poziom hałasu	Brak przekroczenia progu głośności przez 45 sekund
CISZA	Kontrola bezruchu	Pozycja głowy/dłoni	Brak gwałtownych zmian w pozycji w czasie zagrożenia

Źródło: opracowanie własne

W przyszłości każdy mikrokrok mógłby być oceniany w uproszczonej skali:

- 0 - brak wykonania lub działanie szkodliwe,
- 1 - wykonanie z pomocą albo po czasie,
- 2 - wykonanie samodzielne i poprawne.

Taka punktacja umożliwiłaby budowę prostego indeksu mikroprogresji: suma punktów z mikrokroków pokazuje, czy użytkownik przechodzi od działania niepewnego do działania samodzielnego.

7. Pętla poznawcza systemu:

SENSE – INTERPRET – DECIDE – ACT – FEEDBACK

Najważniejszym elementem uzasadnienia projektu jako technologii kognitywnej jest opis pętli poznawczej. *CrisisReady* nie działa jak statyczny film VR. System zbiera dane o użytkowniku, interpretuje je w kontekście scenariusza, decyduje, czy mikrokrok został wykonany poprawnie, uruchamia kolejne zdarzenie i przekazuje użytkownikowi informację zwrotną. Taka pętla pozwala traktować aplikację jako system wspierający decyzje i zachowania, a nie wyłącznie jako środowisko wizualne.

Tabela 6. Realizacja pętli poznawczej

Etap pętli	Realizacja w CrisisReady	Znaczenie kognitywne
SENSE	hand tracking, mikrofon, pozycja głowy, passthrough/VR	system odbiera sygnały związane z percepcją, ruchem i zachowaniem
INTERPRET	detektory mikrokroków, Wit.ai, progowanie ciszy	system interpretuje zachowanie jako poprawne, częściowe lub błędne
DECIDE	ScenarioRunner, MicrostepProgressor, SessionFlowManager	system wybiera dalszy przebieg scenariusza na podstawie danych
ACT	zmiana aktu, dialog dyspozytora, jumpscare, ekran końcowy	system uruchamia konsekwencję działania użytkownika
FEEDBACK	komunikaty, dźwięki, konsekwencje, wynik	użytkownik otrzymuje informację wzmacniającą uczenie

Źródło: opracowanie własne

Warto podkreślić, że feedback w *CrisisReady* nie jest długi ani nadmiernie opisowy. W sytuacji kryzysowej użytkownik nie ma zasobów poznawczych na czytanie rozbudowanych instrukcji. Feedback powinien być krótki, jednoznaczny i powiązany z aktualnym działaniem. Tę zasadę zastosowano w projekcie menu, treningu i komunikatów końcowych.

8. UX/UI i projektowanie interakcji kryzysowej

Interfejs **CrisisReady** został zaprojektowany zgodnie z zasadą: najpierw działanie, potem wyjaśnienie. Oznacza to, że w trakcie sytuacji kryzysowej system nie powinien zasypywać użytkownika instrukcjami, ikonami ani analizą wyników. Zadaniem UI jest ograniczenie obciążenia poznawczego i umożliwienie szybkiego rozpoznania, co należy zrobić, dlatego najważniejsze komunikaty są krótkie, a ćwiczenia nazwano prostymi hasłami: KIEROWNICA, MOWA i CISZA.

Projekt UI obejmował ekran startowy z nazwą **CrisisReady**, przyciski GRAJ i TRENING, ekran ukończenia treningu, komunikaty ćwiczeń oraz ekran przegranej z przyciskiem SPRÓBUJ PONOWNIE. W późniejszych iteracjach usunięto część zbędnych elementów graficznych i uproszczono napisy, aby interfejs był bardziej czytelny.

Tabela 7. Zasady UX/UI i ich zastosowanie w projektowaniu interakcji kryzysowej

Zasada UX/UI	Zastosowanie w projekcie	Uzasadnienie
Minimalizm komunikatów	Krótkie nazwy modułów i interakcje	Redukcja obciążenia poznawczego
Jednoznaczność sygnałów	Każdy etap ma jasny cel	Mniejsze ryzyko pomylenia reakcji
Feedback kontekstowy	Reakcja systemu następuje po działaniu	Uczenie się przez konsekwencję
Dostępność interakcji	Sterowanie dłońmi, brak kontrolerów	Większa naturalność, ale także większe wyzwania techniczne
Tryb treningowy	Osobny tryb przed grą główną	Scaffolding i przygotowanie użytkownika

Źródło: opracowanie własne

9. Architektura techniczna

Architektura systemu została zaprojektowana modułowo.

- Warstwa wejściowa odbiera dane z gogli, mikrofonu i systemu śledzenia dłoni.
- Warstwa interpretacyjna sprawdza, czy dane odpowiadają oczekiwanemu mikrokrokowi.
- Warstwa scenariusza sekwencjonuje akty i przejścia.
- Warstwa feedbacku uruchamia odpowiednie reakcje, a warstwa badawcza zapisuje logi do późniejszej analizy.



Rysunek 3. Diagramy architektury technicznej systemu

Kluczowe fragmenty kodu i ich rola w systemie:

W projekcie nie wklejano do raportu całego kodu źródłowego, ponieważ pełna implementacja znajduje się w repozytorium GitHub. W dokumentacji wybrano najważniejsze fragmenty logiczne, które pokazują, w jaki sposób **CrisisReady** działa jako system kognitywny: odbiera dane od użytkownika, interpretuje je, podejmuje decyzję o przebiegu scenariusza, uruchamia feedback i zapisuje dane badawcze. Najważniejsze fragmenty kodu dotyczą sterowania scenariuszem, detekcji mikrokroków, rozpoznawania mowy, kontroli ciszy oraz logowania wyników.

ScenarioRunner odpowiada za sterowanie przebiegiem scenariusza przez kolejne mikrokroki. Łączy konfigurację scenariusza z mechanizmem postępu i aktywuje odpowiedni detektor dla danego etapu. Dzięki temu system nie jest tylko odtwarzaną animacją, ale reaguje na to, czy użytkownik wykonał wymagane działanie. Kod przyjmuje informacje o aktualnym kroku i stanie detektorów, a następnie decyduje, czy można przejść dalej, czy należy zatrzymać

użytkownika na danym etapie. Potencjalnym miejscem błędu jest niepoprawne przypisanie detektora do mikrokroku albo zbyt szybkie przejście do kolejnej fazy scenariusza.

MicrostepProgressor kontroluje postęp użytkownika w ramach mikrokroków. Jego rola polega na przechowywaniu informacji o tym, które kroki zostały rozpoczęte, ukończone lub niezaliczone. Ten komponent jest ważny z perspektywy mikroprogresji, ponieważ pozwala traktować złożone zadanie jako sekwencję mniejszych jednostek, a nie jako jeden niepodzielny scenariusz. Dane wejściowe pochodzą z detektorów zachowania, a wynikiem działania jest aktualizacja stanu scenariusza. Błędy mogą pojawić się wtedy, gdy mikrokrok zostanie zaliczony zbyt łatwo, zbyt późno albo bez faktycznego wykonania działania przez użytkownika.

SteeringRecoveryDetector odpowiada za sprawdzanie reakcji użytkownika w module poślizgu samochodem. Detektor analizuje, czy użytkownik wykonał odpowiedni ruch dłonią lub kierownicą w momencie utraty kontroli nad pojazdem. Jest to kluczowy fragment kodu dla modułu KIEROWNICA, ponieważ przekłada dane z hand trackingu na decyzję, czy mikrokrok motoryczny został wykonany poprawnie. Dane wejściowe obejmują pozycję dłoni, gest lub obrót kierownicy, a wynikiem jest zaliczenie albo niezaliczenie reakcji. Problemem technicznym może być niestabilne śledzenie dłoni, opóźnienie reakcji albo zbyt restrykcyjny próg zaliczenia ruchu.

WitDictationDetector obsługuje moduł MOWA, czyli rozpoznawanie wypowiedzi użytkownika podczas symulacji rozmowy z numerem alarmowym 112. Jego zadaniem jest odebranie sygnału z mikrofonu, przekazanie wypowiedzi do mechanizmu rozpoznawania mowy oraz sprawdzenie czy użytkownik wypowiedział wymagane słowa kluczowe lub formułkę alarmową. Dzięki temu system może ocenić nie tylko obecność głosu, ale także przybliżoną treść komunikatu. Dane wejściowe pochodzą z mikrofonu, a wynikiem jest informacja czy wypowiedź spełniła kryteria mikrokroku. Potencjalne błędy mogą wynikać z szumu otoczenia, niewyraźnej wymowy, błędnej transkrypcji albo zbyt wąsko dobranych słów kluczowych.

SilenceMicrostepDetector odpowiada za moduł CISZA. Kod kontroluje poziom głośności z mikrofonu i sprawdza, czy użytkownik pozostaje poniżej ustalonego progu hałasu przez wymagany czas. Jeśli próg zostanie przekroczony, system może naliczyć ostrzeżenie, czyli strike, a po przekroczeniu dopuszczalnej liczby błędów zakończyć mikrokrok porażką. Ten

fragment kodu jest ważny, ponieważ zamienia zachowanie użytkownika, czyli ciszę lub hałas, na mierzalny wskaźnik samoregulacji. Dane wejściowe obejmują aktualny poziom głośności, próg hałasu i czas trwania ciszy, a wynikiem jest sukces, postęp albo porażka mikrokroku. W wersji demonstracyjnej czas i próg ciszy powinny być traktowane jako parametry konfigurowalne, ponieważ różne pomieszczenia mogą mieć różny poziom szumu tła.

JSONLogger odpowiada za zapis danych badawczych do pliku metrics.jsonl. Rejestruje m.in. rozpoczęcie i zakończenie sesji, znaczniki czasu, identyfikator sesji, typ zdarzenia oraz dodatkowe dane przekazane przez poszczególne moduły. Jest to kluczowy komponent z perspektywy walidacji projektu, ponieważ umożliwia późniejszą analizę przebiegu sesji, czasu reakcji, liczby błędów i ukończenia mikrokroków. Dane wejściowe pochodzą z różnych części systemu, a wynikiem jest kolejny wpis w pliku logów. Potencjalnym problemem może być brak dostępu do zapisu pliku, przerwanie aplikacji przed zamknięciem sesji albo niepełne logowanie zdarzeń.

W projekcie istotną rolę pełniły także skrypty edytorowe odpowiedzialne za budowanie sceny z kodu. **LabRoomBuilder**, **ActsBuilder** i **MetaSceneBuilder** pozwalały generować pomieszczenie, akty scenariusza oraz podstawowe elementy środowiska bez ręcznego układania całej sceny w edytorze Unity. Dzięki temu projekt był bardziej powtarzalny i łatwiejszy do odtworzenia po zmianach w strukturze sceny. Ten element zwiększa replikowalność projektu, ponieważ osoba uruchamiająca repozytorium może przebudować scenę według tej samej procedury. Ryzykiem jest jednak to, że po zmianie skryptów budujących scenę trzeba pamiętać o jej ponownym wygenerowaniu, inaczej poprawki mogą nie pojawić się w końcowym buildzie.

Opisane fragmenty kodu pokazują, że **CrisisReady** ma strukturę modułową. Detektory odpowiadają za interpretowanie konkretnych zachowań użytkownika, komponenty scenariusza sterują przebiegiem aktów, a logger zapisuje dane do późniejszej analizy. Dzięki temu system realizuje założenie pętli poznawczej SENSE – INTERPRET – DECIDE – ACT – FEEDBACK: odbiera sygnały z dłoni, głowy i mikrofonu, interpretuje je w kontekście mikrokroku, podejmuje decyzję o sukcesie lub porażce, uruchamia konsekwencje scenariusza i zapisuje wynik działania.

10. Wykorzystane technologie, narzędzia i zasoby

Projekt został przygotowany w Unity 6 LTS, wersja 6000.0.74f1. Build przygotowywano na platformę Android dla Meta Quest 3/3S, z wykorzystaniem backendu IL2CPP oraz architektury ARM64. W projekcie wykorzystano także XR Interaction Toolkit, OpenXR, Meta XR SDK Core oraz pakiet com.unity.xr.hands. Pakiet aplikacji został skonfigurowany jako com.grupa1.mrcrisistrainer. W projekcie wykorzystano także passthrough, mikrofon, rozpoznawanie mowy Wit.ai oraz głos dyspozytora wygenerowany przy użyciu syntezy mowy Microsoft Paulina.

Tabela 8. Wykorzystane technologie, narzędzia i zasoby projektu

Kategoria	Technologia / zasób	Zastosowanie
Silnik	Unity 6 LTS	implementacja scen, logiki i build APK
Język	C#	skrypty, detektory, zarządzanie scenariuszem
XR	OpenXR, XR Hands, Meta XR SDK	śledzenie dłoni, passthrough, obsługa Quest
Sprzęt	Meta Quest 3/3S	uruchomienie aplikacji MR/VR
Głos	Microphone + Wit.ai	rozpoznawanie formułek w module 112
Audio	Microsoft Paulina TTS	głos dyspozytora
Modele	Mixamo, Sketchfab, Itch.io	intruz, auto, pokój, ciężarówka
Dźwięki	Freesound.org	telefon, syreny, uderzenie, dźwięki grozy
Analiza	JASP	planowana analiza danych z badania
Logowanie	JSONL	zapis zdarzeń, czasów i wyników

11. Funkcjonalność programu

Program realizuje funkcje podstawowe, kognitywne, badawcze i administracyjno-techniczne:

- Funkcje podstawowe – uruchomienie aplikacji, wybór trybu, przejście sesji, restart po przegranej i odtworzenie dialogów.
- Funkcje kognitywne – wsparcie decyzji i zachowania poprzez mikro kroki, feedback oraz trening pod presją.
- Funkcje badawcze – logowanie zdarzeń i przygotowanie do kwestionariuszy.
- Funkcje techniczne – budowanie scen z kodu, konfigurację manifestu Androida i generowanie APK.

Tabela 9. Typy funkcji realizowanych przez program

Typ funkcji	Przykłady w CrisisReady
Podstawowe	menu, GRAJ, TRENING, restart, przejście pełnej sesji
Kognitywne	trening reakcji, kontrola ciszy, jasna komunikacja, krótkie feedbacki
Sensoryczne	hand tracking, detekcja pozycji głowy, mikrofon, passthrough
Decyzyjne	zaliczenie lub niezaliczenie mikrokroku, przejście do kolejnej fazy
Feedback	głos dyspozytora, dźwięki, jumpscare, ekran końcowy
Badawcze	metrics.jsonl, znaczniki czasu, wyniki, próby, błędy
Techniczne	builder scen, build APK, konfiguracja OpenXR i AndroidManifest

Źródło: opracowanie własne

12. Logowanie danych i przygotowanie walidacji

Jednym z wymagań projektu było przygotowanie systemu do pomiaru efektywności. **CrisisReady** zapisuje metryki do pliku `metrics.jsonl`. Dane te mogą obejmować znaczniki czasu, nazwę aktu, nazwę mikrokroku, rezultat wykonania, liczbę prób, błędy i ukończenie scenariusza. Taki format jest prosty, czytelny i możliwy do późniejszej analizy w arkuszu kalkulacyjnym, Pythonie lub JASP.

Tabela 10. Zmienne zapisywane w logach `metrics.jsonl` i ich znaczenie badawcze

Zmienna w logach	Znaczenie badawcze
<code>participant_id</code>	pseudonimowy identyfikator uczestnika
<code>session_id</code>	numer lub kod sesji badawczej
<code>timestamp</code>	czas zdarzenia
<code>scenario_phase</code>	akt lub moduł scenariusza
<code>microstep_id</code>	konkretny mikrokrok
<code>result</code>	sukces, błąd, przekroczenie czasu, przerwanie
<code>response_time_ms</code>	czas reakcji
<code>hints_count</code>	liczba podpowiedzi
<code>noise_threshold_exceeded</code>	czy przekroczono próg ciszy
<code>speech_keywords_detected</code>	czy wykryto wymagane słowa kluczowe

Źródło: opracowanie własne

13. Proces wytwórczy, harmonogram i metodyka pracy

Prace prowadzono iteracyjnie. Zaczęliśmy pracę od analizy wymagań i artykułu wzorcowego, następnie przygotowaliśmy wstępną koncepcję scenariusza (zdjęcie poniżej), projekt UI i mechanikę mikrokroków. Kolejne etapy obejmowały implementację scen, integrację hand trackingu, rozpoznawania mowy, assetów i logowania, a następnie testowanie na fizycznych goglach Quest 3/3S. Ze względu na specyfikę VR/MR każda istotna zmiana wymagała budowania aplikacji, wgrania jej na gogle i sprawdzenia w rzeczywistych warunkach.

Tabela 11. Harmonogram etapów pracy nad projektem

Etap	Zakres prac	Rezultat
1. Koncepcja	analiza problemu, wybór scenariuszy, decyzja o mikroprogresji	opis celu i głównych modułów
2. UX/UI	menu, trening, komunikaty, nazwy modułów	czytelne ekrany i uproszczony interfejs
3. Implementacja bazowa	sceny, akty, sterowanie, podstawowe przejścia	minimalny prototyp
4. Integracja MR/VR	passthrough, VR, hand tracking, mikrofon	działanie na Quest 3/3S
5. Integracja narracji	wypadek, intruz, telefon, syreny, dialog	spójny scenariusz fabularny
6. Testy i poprawki	debugowanie modeli, UI, wydajności i dłoni	grywalna wersja od początku do końca
7. Dokumentacja	opis systemu, role, problemy, bibliografia	sprawozdanie i materiały do oceny

Źródło: opracowanie własne

Wspólny postęp był śledzony dzięki harmonogramowi w aplikacji Notion, gdzie każdy mógł kontrolować listę swoich zadań oraz termin na ich wykonanie, a także mógł zaznaczać, które z nich udało się już wykonać. Efekty swoich prac zamieszczaliśmy we wspólnym dysku Google – każdy miał swój folder, w którym umieszczał wykonane pliki, znalezione informacje czy linki oraz wszystkie potrzebne materiały. W ten sposób jako zespół, byliśmy świadomi postępu, jaki jest dokonywany oraz listy zadań, które wymagają jeszcze pracy.



Rysunek 4. Wstępna koncepcja scenariusza projektu opracowana przez zespół

Metodyka pracy była zbliżona do podejścia przyrostowego. Zamiast czekać na pełną implementację, testowaliśmy małe przyrosty. Dzięki temu błędy techniczne były wykrywane względnie wcześnie, choć część problemów, takich jak śledzenie dłoni lub pozycjonowanie assetów, wymagała wielu iteracji.

Pierwszym przyrostem było przygotowanie podstawowej struktury aplikacji. Na tym etapie powstało menu startowe, przyciski GRAJ i TRENING oraz podstawowa logika przechodzenia między ekranami. Celem tego etapu było stworzenie stabilnego punktu wejścia do aplikacji, z którego użytkownik mógł rozpocząć trening lub scenariusz główny.

Drugim przyrostem był tryb treningowy, oparty na trzech prostych ćwiczeniach: KIEROWNICA, MOWA i CISZA. W tej części skupiono się na opracowaniu krótkich, jednoznacznych instrukcji oraz na przetestowaniu, czy użytkownik rozumie, jakie działanie ma wykonać. Tryb treningowy pełnił funkcję scaffoldingu, ponieważ przygotowywał użytkownika do późniejszego przejścia przez pełny scenariusz kryzysowy.

Kolejnym etapem była implementacja modułu poślizgu samochodem. W tym przyroście przygotowano scenę jazdy, kierownicę, reakcję na poślizg oraz przejście do zdarzenia wypadku.

Był to pierwszy mikrokrok o charakterze motorycznym, w którym użytkownik musiał zareagować ruchem dłoni w określonym momencie.

Następnie rozwijano moduł ukrycia. Na tym etapie przygotowano scenę pokoju, pozycję pod łóżkiem oraz mechanizm sprawdzający, czy użytkownik przyjął odpowiednią pozycję. Ponieważ projekt zakładał tryb siedzący, konieczne było dostosowanie interakcji do ograniczeń przestrzennych użytkownika. Zamiast wymagania realnego wejścia pod łóżko, zastosowano gest i pozycję głowy jako uproszczony wskaźnik ukrycia.

Kolejny przyrost dotyczył modułu telefonu alarmowego 112. W tej części zintegrowano mikrofon, dialog z dyspozytorem, rozpoznawanie wypowiedzi oraz głos wygenerowany za pomocą syntezy mowy. Celem było stworzenie sytuacji, w której użytkownik nie tylko widzi zagrożenie, ale musi także sformułować komunikat alarmowy w sposób zrozumiały i uporządkowany.

Następnie zaimplementowano moduł ciszy. Obejmował on detekcję poziomu hałasu, warunek zachowania ciszy przez określony czas oraz konsekwencje przerwania zadania. Ten etap był szczególnie istotny z perspektywy kognitywnej, ponieważ dotyczył samoregulacji, hamowania reakcji i kontroli zachowania w sytuacji zagrożenia.

Ostatnie przyrosty obejmowały logowanie danych, przygotowanie metryk do późniejszej walidacji oraz testy techniczne. Dodano zapis zdarzeń do pliku JSONL, co umożliwia późniejszą analizę czasu reakcji, liczby błędów, ukończenia mikrokroków i przebiegu sesji. Równolegle prowadzono testy buildów APK na gościach, poprawiano modele 3D, interfejs, wydajność oraz działanie śledzenia dłoni i mikrofonu.

Dzięki pracy przyrostowej możliwe było ograniczenie ryzyka technicznego. Każdy moduł był najpierw sprawdzany jako osobny element, a dopiero później integrowany z pozostałymi częściami scenariusza. Pozwoliło to stopniowo przejść od prostego menu i pojedynczych ćwiczeń do grywalnego prototypu, w którym użytkownik może przejść pełną sekwencję: jazda, poślizg, wypadek, ukrycie, telefon 112, cisza oraz zakończenie scenariusza.

14. Podział obowiązków w zespole

Projekt został zrealizowany przez pięcioosobowy zespół. Podział obowiązków obejmował: implementację mechaniki, testowanie, scenariusze dialogowe, dostarczanie assetów, oprawę graficzną, przygotowanie kwestionariuszy i analizę artykułu źródłowego.

- Jakub Cieniuch – mechanika gry, praca z Claude Code, assety, debugowanie, testy, implementacja logiki i integracja wielu komponentów
- Wiktoria Bartek – testowanie gry i feedback, wykrywanie błędów, sprawdzanie czytelności doświadczenia, przygotowanie końcowej prezentacji projektu
- Oliwia Frueauff – scenariusze i formułki dyspozytora 112, opracowanie treści komunikacji kryzysowej, opracowanie testów wiedzy związanych z poprawnym zachowaniem podczas poślizgu oraz resuscytacji (pierwotna wersja scenariusza) na podstawie oficjalnych wymogów
- Magdalena Grzesiak – modele 3D i dźwięki, dostarczanie zasobów wizualnych i audio
- Adriana Jankowiak – oprawa graficzna menu i treningu, kwestionariusze, analiza *MR.Drum*, warstwa UI, badawcza i teoretyczna, opracowanie sprawozdania

Dodatkowo każdy uczestnik miał aktywny twórczy wkład w projekt - od początku do końca brał udział w dyskusjach i burzach mózgu, dzielił się pomysłami i wpływał na końcowy efekt programu.

15. Testowanie techniczne

Tabela 12. Zakres testowania technicznego prototypu

Obszar testów	Co sprawdzić	Kryterium akceptacji
Wydajność	FPS, spadki klatek, płynność renderowania	stabilne działanie bez zauważalnych przycięć
Opóźnienia	latency gest - reakcja, latency audio	feedback pojawia się wystarczająco szybko
Hand tracking	utrata śledzenia, jitter, warunki oświetlenia	dłonie są wykrywane w typowych warunkach pokoju
Mikrofon	uprawnienia, poziom szumu, wykrywanie ciszy	system poprawnie odróżnia ciszę od hałasu
Rozpoznawanie mowy	Wit.ai, słowa kluczowe, błędy transkrypcji	formułki 112 są rozpoznawane w większości prób
Pozycjonowanie assetów	skala, obrót, zapadanie w podłoże	modele są widoczne i nie psują immersji
Bezpieczeństwo	możliwość przzerwania, seated mode	użytkownik nie musi fizycznie chodzić ani ryzykować kolizji
Logowanie	kompletność metrics.jsonl	po sesji można odtworzyć przebieg zadania

Źródło: opracowanie własne

W wyniku testów potwierdzono, że w prototypie aplikacja uruchamia się na fizycznych goglach Meta Quest 3/3S, a użytkownik może przejść główną sekwencję scenariusza: menu, trening, jazdę, poślizg, ukrycie, telefon 112, moduł ciszy oraz zakończenie sukcesem lub porażką. Poprawnie działały także podstawowe przejścia między aktami, odtwarzanie dźwięków oraz ekran końcowy.

Testy ujawniły jednak ograniczenia związane z hand trackingiem, mikrofonem, pozycjonowaniem modeli 3D oraz wydajnością urządzenia. Śledzenie dłoni było mniej stabilne przy gorszym oświetleniu lub szybkich ruchach, dlatego interakcje musiały zostać uproszczone. Moduł mikrofonu działał jako mechanizm demonstracyjny, ale wymagałby dalszej kalibracji progów hałasu i rozpoznawania mowy. Testy wykazały również, że każda zmiana w projekcie wymagała ponownego buildu i instalacji APK, co znacząco wydłużało iteracje.

16. Propozycja badania użytkowników i procedura walidacji

Proponowana próba pilotażowa mogłaby obejmować 8-10 osób, co wystarcza do wykrycia głównych problemów UX i technicznych. W pełniejszym badaniu warto porównać grupę korzystającą z mikroprogresji MR/VR z grupą kontrolną, która otrzymuje klasyczny materiał szkoleniowy lub wersję VR bez adaptacyjnego feedbacku. Taki układ pozwalałby sprawdzić, czy przewagę daje sama immersja, czy właśnie pętla poznawcza i mikroprogresja.

Tabela 13. Proponowana procedura badania użytkowników i dane zbierane na kolejnych etapach

Etap badania	Opis	Dane zbierane
Pre-test	uczestnik wykonuje uproszczone zadanie bez pełnego wsparcia	czas, błędy, liczba pominięć
Trening	uczestnik przechodzi tryb TRENING i sesję <i>CrisisReady</i>	czas mikrokroków, odpowiedzi, błędy
Post-test	uczestnik powtarza zadanie z mniejszą liczbą odpowiedzi	różnica względem pre-testu
Kwestionariusze	NASA-TLX, SUS, SSQ	obciążenie, użyteczność, cybersickness
Wywiad	krótkie pytania o trudność i zrozumiałość	jakościowe wyjaśnienie problemów
Retencja	powtórka po kilku dniach	utrzymanie efektu
Transfer	zmieniony wariant scenariusza	czy umiejętność nie jest tylko zapamiętaniem scenki

Zródło: opracowanie własne

17. Metryki efektywności, UX i obciążenia poznawczego

W procedurze walidacji zaproponowaliśmy wykorzystanie trzech narzędzi kwestionariuszowych:

- NASA-TLX do oceny obciążenia poznawczego i frustracji,
- SUS do oceny użyteczności systemu,
- SSQ do kontroli objawów cybersickness.

Zrezygnowano z osobnego pomiaru poczucia obecności, ponieważ głównym celem badania nie jest ocena immersji samego środowiska MR/VR, lecz skuteczność treningu, zrozumiałość interfejsu i bezpieczeństwo użytkownika.

Metryki zostały dobrane tak, aby sprawdzić nie tylko to, czy użytkownik ukończył scenariusz, ale także jak szybko reagował, ile popełnił błędów, czy zrozumiał interfejs oraz jakim kosztem poznawczym i fizjologicznym wykonał zadanie. W projekcie szczególnie istotne było połączenie danych obiektywnych, zapisywanych przez system, z subiektywną oceną użytkownika po zakończeniu sesji.

Tabela 14. Miary efektywności, UX i obciążenia poznawczego

Obszar	Metryka	Źródło danych	Po co mierzymy
Efektywność działania	Czas reakcji na poślizg, czas ukończenia mikrokroku	Metrics.jsonl	Sprawdzenie szybkości decyzji i wykonania reakcji
Poprawność wykonania	Liczba błędnych mikrokroków, liczba powtórzeń, liczba nieudanych etapów	Logi systemowe	Identyfikacja najtrudniejszych modułów i momentów scenariusza
Postęp użytkownika	Wynik mikrokroków w skali 0-2	Logi systemowe	Ocena czy użytkownik przechodzi od działania niepewnego do samodzielnego
Obciążenie poznawcze	NASA-TLX	Ankieta po sesji	Ocena wysiłku mentalnego, presji czasu, trudności zadania i frustracji
Użyteczność systemu	SUS	Ankieta UX po sesji	Sprawdzenie czy system jest zrozumiały, spójny i możliwy do użycia bez nadmiernego wsparcia
Komfort VR/MR	SSQ	Ankieta po sesji	Kontrola objawów cybersickness
Zachowanie użytkownika	Obserwacja przebiegu sesji	Notatki	Sprawdzenie, gdzie użytkownik się waha

Źródło: opracowanie własne

Oprócz przygotowania procedury walidacji przeprowadzono krótką ewaluację wewnętrzną prototypu na trzech członkach zespołu. Miała ona charakter pilotażowy i techniczno-UX, dlatego nie była traktowana jako pełne badanie użytkowników ani jako podstawa do weryfikacji hipotez badawczych. Celem było sprawdzenie, czy wybrane kwestionariusze są możliwe do zastosowania po sesji z prototypem oraz czy pozwalają zebrać informacje dotyczące użyteczności, obciążenia poznawczego i komfortu korzystania z aplikacji.

Po przejściu scenariusza uczestnicy wypełnili krótkie narzędzia ewaluacyjne obejmujące ocenę użyteczności systemu, obciążenia poznawczego oraz ewentualnego dyskomfortu związanego z użyciem gogli VR/MR. Wykorzystano w tym celu założenia kwestionariuszy SUS, NASA-TLX oraz SSQ, które zostały opisane jako główne narzędzia planowanej walidacji. Zebrane odpowiedzi miały pomóc w ocenie, czy interfejs jest zrozumiały, czy zadania nie powodują nadmiernej frustracji oraz czy sesja nie wywołuje wyraźnych objawów cybersickness.

Odpowiedzi 1 – Oliwia

NASA-TLX Mental Workload Rating Scale

Please place an "X" along each scale at the point that best indicates your experience with the display configuration.

Mental Demand: How much mental and perceptual activity was required (e.g., thinking, deciding, calculating, remembering, looking, searching, etc)? Was the mission easy or demanding, simple or complex, exacting or forgiving?

Low High

Physical Demand: How much physical activity was required (e.g., pushing, pulling, turning, controlling, activating, etc.)? Was the mission easy or demanding, slow or brisk, slack or strenuous, restful or laborious?

Low High

Temporal Demand: How much time pressure did you feel due to the rate or pace at which the mission occurred? Was the pace slow and leisurely or rapid and frantic?

Low High

Performance: How successful do you think you were in accomplishing the goals of the mission? How satisfied were you with your performance in accomplishing these goals?

Low High

Effort: How hard did you have to work (mentally and physically) to accomplish your level of performance?

Low High

Frustration: How discouraged, stressed, irritated, and annoyed versus gratified, relaxed, content, and complacent did you feel during your mission?

Low High

Total Un-weighted Mental Workload Score: 56,7

- To calculate an un-weighted score, average the raw data scores from the 6 factors.
- To calculate a weighted score, add the number of times a factor was selected on the ranking page. Divide that number by 15. Multiply it by the factor score. Add all ranking*rating combination to get the total weighted score.

SUS składa się 10 pytań oraz 5 stopniowej skali ocen (opartej o skalę Likerta) :

- Będę często korzystać z systemu. **2**
- System jest niepotrzebnie skomplikowany. **4**
- System jest łatwy w użyciu. **4**
- Będę potrzebował wsparcia technicznego, aby korzystać z systemu. **4**
- Różne funkcje systemu są łatwo dostępne. **3**
- W systemie jest zbyt wiele niespójności. **4**
- Większość osób będzie w stanie opanować system bardzo szybko. **4**
- System jest kłopotliwy w użyciu. **4**
- Czuję się bardzo pewnie korzystając z systemu. **5**
- Musieliem opanować wiele rzeczy przed rozpoczęciem pracy z systemem. **3**

	1	2	3	4	5	6	7		
irytujący	☐	☐	☐	☐	☐	☒	☐	przyjemny	1
niezrozumiały	☐	☐	☐	☐	☐	☒	☐	zrozumiały	2
kreatywny	☒	☐	☐	☐	☐	☐	☐	nudny	3
łatwy do nauki	☐	☒	☐	☐	☐	☐	☐	trudny do nauki	4
wartościowy	☐	☒	☐	☐	☐	☐	☐	gorszy	5
nudny	☐	☐	☐	☐	☐	☒	☐	ekscytujący	6
nieinteresujący	☐	☐	☐	☐	☐	☒	☐	interesujący	7
nieprzewidywalny	☒	☐	☐	☐	☐	☐	☐	przewidywalny	8
szybki	☐	☐	☒	☐	☐	☐	☐	wolny	9
pomysłowy	☒	☐	☐	☐	☐	☐	☐	konwencjonalny	10
przeszkadzający	☐	☐	☐	☐	☐	☒	☐	wspierający	11
dobry	☒	☐	☐	☐	☐	☐	☐	zły	12
skomplikowany	☐	☐	☒	☐	☐	☐	☐	łatwy	13
nieprzyjemny	☐	☐	☐	☐	☒	☐	☐	przyjemny	14
zwykły	☐	☐	☐	☒	☐	☐	☐	skrajny	15
niemiły	☐	☐	☐	☐	☒	☐	☐	miły	16
bezpieczny	☒	☐	☐	☐	☐	☐	☐	niebezpieczny	17
motywujący	☒	☐	☐	☐	☐	☐	☐	demotywujący	18
spełnia oczekiwania	☐	☒	☐	☐	☐	☐	☐	nie spełnia oczekiwań	19
nieefektywny	☐	☐	☐	☐	☐	☒	☐	efektywny	20
przejrzysty	☐	☐	☒	☐	☐	☐	☐	zagałowany	21
niepraktyczny	☐	☐	☐	☐	☐	☒	☐	praktyczny	22
zorganizowany	☒	☐	☐	☐	☐	☐	☐	zagrażony	23
atrakcyjny	☒	☐	☐	☐	☐	☐	☐	nieatrakcyjny	24
przyjazny	☒	☐	☐	☐	☐	☐	☐	nieprzyjazny	25
konserwatywny	☐	☐	☐	☐	☐	☒	☐	innowacyjny	26

Odpowiedzi 2 – Wiktoria

NASA-TLX Mental Workload Rating Scale

Please place an "X" along each scale at the point that best indicates your experience with the display configuration.

Mental Demand: How much mental and perceptual activity was required (e.g., thinking, deciding, calculating, remembering, looking, searching, etc)? Was the mission easy or demanding, simple or complex, exacting or forgiving?

Low
Low
High
 60

Physical Demand: How much physical activity was required (e.g., pushing, pulling, turning, controlling, activating, etc)? Was the mission easy or demanding, slow or brisk, slack or strenuous, restful or laborious?

Low
Low
High
 20

Temporal Demand: How much time pressure did you feel due to the rate or pace at which the mission occurred? Was the pace slow and leisurely or rapid and frantic?

Low
Low
High
 55

Performance: How successful do you think you were in accomplishing the goals of the mission? How satisfied were you with your performance in accomplishing these goals?

Low
Low
High
 100

Effort: How hard did you have to work (mentally and physically) to accomplish your level of performance?

Low
Low
High
 55

Frustration: How discouraged, stressed, irritated, and annoyed versus gratified, relaxed, content, and complacent did you feel during your mission?

Low
Low
High
 35

Total Un-weighted Mental Workload Score¹: 54,17

1. To calculate an un-weighted score, average the raw data scores from the 6 factors.
2. To calculate a weighted score, add the number of times a factor was selected on the ranking page. Divide that number by 15. Multiply it by the factor score. Add all ranking*rating combination to get the total weighted score.

	1	2	3	4	5	6	7		
irytujący	☐	☐	☐	☐	☐	☒	☐	przyjemny	1
niezrozumiały	☐	☐	☐	☐	☐	☒	☐	zrozumiały	2
kreatywny	☒	☐	☐	☐	☐	☐	☐	nudny	3
łatwy do nauki	☐	☒	☐	☐	☐	☐	☐	trudny do nauki	4
wartościowy	☒	☐	☐	☐	☐	☐	☐	gorzki	5
nudny	☐	☐	☐	☐	☐	☐	☒	ekscytujący	6
nieinteresujący	☐	☐	☐	☐	☐	☐	☒	interesujący	7
nieprzewidywalny	☒	☐	☐	☐	☐	☐	☐	przewidywalny	8
szybki	☒	☐	☐	☐	☐	☐	☐	wolny	9
pomysłowy	☒	☐	☐	☐	☐	☐	☐	konwencjonalny	10
przeszkadzający	☐	☐	☐	☐	☒	☐	☐	wspierający	11
dobry	☒	☐	☐	☐	☐	☐	☐	zły	12
skomplikowany	☐	☐	☒	☐	☐	☐	☐	łatwy	13
nieprzyjemny	☐	☐	☐	☐	☒	☐	☐	przyjemny	14
zwykły	☐	☐	☐	☒	☐	☐	☐	skrajny	15
niemiły	☐	☐	☐	☐	☐	☒	☐	miły	16
bezpieczny	☐	☒	☐	☐	☐	☐	☐	niebezpieczny	17
motywujący	☒	☐	☐	☐	☐	☐	☐	demotywujący	18
spełnia oczekiwania	☒	☐	☐	☐	☐	☐	☐	nie spełnia oczekiwań	19
nieefektywny	☐	☐	☐	☐	☐	☒	☐	efektywny	20
przejrzysty	☐	☒	☐	☐	☐	☐	☐	zagałwany	21
niepraktyczny	☐	☐	☐	☐	☐	☒	☐	praktyczny	22
zorganizowany	☒	☐	☐	☐	☐	☐	☐	zagałwany	23
atrakcyjny	☒	☐	☐	☐	☐	☐	☐	nieatrakcyjny	24
przyjazny	☐	☒	☐	☐	☐	☐	☐	nieprzyjazny	25
konserwatywny	☐	☐	☐	☐	☐	☒	☐	innowacyjny	26

SUS składa się 10 pytań oraz 5 stopniowej skali ocen (opartej o skalę Likerta) :

1. Będę często korzystać z systemu. 2
2. System jest niepotrzebnie skomplikowany. 1
3. System jest łatwy w użyciu. 4
4. Będę potrzebował wsparcia technicznego, aby korzystać z systemu. 1
5. Różne funkcje systemu są łatwo dostępne. 4
6. W systemie jest zbyt wiele niespójności. 1
7. Większość osób będzie w stanie opanować system bardzo szybko. 5
8. System jest kłopotliwy w użyciu. 1
9. Czuję się bardzo pewnie korzystając z systemu. 3
10. Musiałem opanować wiele rzeczy przed rozpoczęciem pracy z systemem. 2

Odpowiedzi 3 – Magdalena

NASA-TLX Mental Workload Rating Scale

Please place an "X" along each scale at the point that best indicates your experience with the display configuration.

Mental Demand: How much mental and perceptual activity was required (e.g., thinking, deciding, calculating, remembering, looking, searching, etc)? Was the mission easy or demanding, simple or complex, exacting or forgiving?

Low
Low
High
 75

Physical Demand: How much physical activity was required (e.g., pushing, pulling, turning, controlling, activating, etc)? Was the mission easy or demanding, slow or brisk, slack or strenuous, restful or laborious?

Low
Low
High
 50

Temporal Demand: How much time pressure did you feel due to the rate or pace at which the mission occurred? Was the pace slow and leisurely or rapid and frantic?

Low
Low
High
 85

Performance: How successful do you think you were in accomplishing the goals of the mission? How satisfied were you with your performance in accomplishing these goals?

Low
Low
High
 75

Effort: How hard did you have to work (mentally and physically) to accomplish your level of performance?

Low
Low
High
 60

Frustration: How discouraged, stressed, irritated, and annoyed versus gratified, relaxed, content, and complacent did you feel during your mission?

Low
Low
High
 30

Total Un-weighted Mental Workload Score¹: 62,5

1. To calculate an un-weighted score, average the raw data scores from the 6 factors.
2. To calculate a weighted score, add the number of times a factor was selected on the ranking page. Divide that number by 15. Multiply it by the factor score. Add all ranking*rating combination to get the total weighted score.

	1	2	3	4	5	6	7		
irytujący	☐	☐	☐	☐	☐	☒	☐	przyjemny	1
niezrozumiały	☐	☐	☐	☐	☐	☒	☐	zrozumiały	2
kreatywny	☒	☐	☐	☐	☐	☐	☐	nudny	3
łatwy do nauki	☐	☒	☐	☐	☐	☐	☐	trudny do nauki	4
wartościowy	☒	☐	☐	☐	☐	☐	☐	gorzki	5
nudny	☐	☐	☐	☐	☐	☐	☒	ekscytujący	6
nieinteresujący	☐	☐	☐	☐	☐	☐	☒	interesujący	7
nieprzewidywalny	☐	☐	☐	☐	☐	☐	☒	przewidywalny	8
szybki	☐	☐	☐	☐	☐	☐	☒	wolny	9
pomysłowy	☒	☐	☐	☐	☐	☐	☐	konwencjonalny	10
przeszkadzający	☐	☐	☐	☐	☐	☒	☐	wspierający	11
dobry	☒	☐	☐	☐	☐	☐	☐	zły	12
skomplikowany	☐	☐	☐	☐	☐	☐	☒	łatwy	13
nieprzyjemny	☐	☐	☐	☐	☐	☒	☐	przyjemny	14
zwykły	☐	☐	☐	☐	☐	☒	☐	skrajny	15
niemiły	☐	☐	☐	☐	☐	☐	☒	miły	16
bezpieczny	☐	☒	☐	☐	☐	☐	☐	niebezpieczny	17
motywujący	☒	☐	☐	☐	☐	☐	☐	demotywujący	18
spełnia oczekiwania	☒	☐	☐	☐	☐	☐	☐	nie spełnia oczekiwań	19
nieefektywny	☐	☐	☐	☐	☐	☒	☐	efektywny	20
przejrzysty	☐	☐	☐	☐	☐	☒	☐	zagałwany	21
niepraktyczny	☐	☐	☐	☐	☐	☒	☐	praktyczny	22
zorganizowany	☒	☐	☐	☐	☐	☐	☐	zagałwany	23
atrakcyjny	☒	☐	☐	☐	☐	☐	☐	nieatrakcyjny	24
przyjazny	☐	☒	☐	☐	☐	☐	☐	nieprzyjazny	25
konserwatywny	☐	☐	☐	☐	☐	☒	☐	innowacyjny	26

Ada

SUS składa się 10 pytań oraz 5 stopniowej skali ocen (opartej o skalę Likerta) :

1. Będę często korzystać z systemu. 3
2. System jest niepotrzebnie skomplikowany. 1
3. System jest łatwy w użyciu. 5
4. Będę potrzebował wsparcia technicznego, aby korzystać z systemu. 2
5. Różne funkcje systemu są łatwo dostępne. 5
6. W systemie jest zbyt wiele niespójności. 1
7. Większość osób będzie w stanie opanować system bardzo szybko. 5
8. System jest kłopotliwy w użyciu. 1
9. Czuję się bardzo pewnie korzystając z systemu. 4
10. Musiałem opanować wiele rzeczy przed rozpoczęciem pracy z systemem. 2

Ze względu na bardzo małą liczebność próby oraz fakt, że uczestnikami byli członkowie zespołu znający cel projektu, wyniki tej ewaluacji należy traktować wyłącznie jako informację pomocniczą. Nie pozwalają one na wyciąganie wniosków o skuteczności mikroprogresji ani na potwierdzenie hipotez badawczych. Na obecnym etapie projektu pytania badawcze nie zostały jeszcze rozstrzygnięte empirycznie, ponieważ *CrisisReady* zostało opracowane jako grywalny prototyp, a nie jako zakończone badanie eksperymentalne. Pilotaż wskazał jednak, że zaproponowane narzędzia mogą zostać wykorzystane w przyszłym badaniu z osobami spoza zespołu, ponieważ projekt zawiera strukturę mikrokroków, logowanie danych do metrics.jsonl, możliwość pomiaru czasu reakcji, błędów i ukończenia etapów oraz propozycję wykorzystania kwestionariuszy NASA-TLX, SUS i SSQ.

W dalszym rozwoju projektu procedura walidacji mogłaby zostać rozszerzona o dodatkowe narzędzia. Można byłoby zastosować IPQ do pomiaru poczucia obecności w środowisku wirtualnym oraz krótką skalę stresu lub pobudzenia, jeżeli projekt zostałby rozwinięty w kierunku bardziej zaawansowanego treningu odporności na presję. Na obecnym etapie uznano jednak, że najważniejsze są trzy obszary: skuteczność treningu, użyteczność interfejsu i bezpieczeństwo użytkownika.

18. Wyzwania etyczne, prywatność i bezpieczeństwo

Projekt dotyczy sytuacji kryzysowych, stresu, głosu, ruchu i zachowania użytkownika, dlatego wymaga osobnej refleksji etycznej. Najważniejsze ryzyka dotyczą prywatności danych, bezpieczeństwa fizycznego, emocjonalnego obciążenia użytkownika, wtórnego użycia logów i nadmiernego zaufania do systemu.

Tabela 15. Ryzyka etyczne, prywatnościowe i bezpieczeństwa oraz sposoby ich minimalizacji

Ryzyko	Opis	Minimalizacja
Dane głosowe	moduł MOWA korzysta z mikrofonu i rozpoznawania mowy	nie zapisywać nagrań, tylko wynik rozpoznania lub słowa kluczowe
Dane behawioralne	logi opisują reakcje, błędy i czas użytkownika	pseudonimizacja participant_id, brak imion i nazwisk w logach
Stres i dyskomfort	scenariusz zawiera wypadek, włamanie i jumpscare	zgoda świadoma, możliwość przerywania, ostrzeżenie przed treścią
Cybersickness	VR/MR może powodować nudności lub dezorientację	SSQ, krótka sesja, seated mode, przerywanie na życzenie
Bezpieczeństwo fizyczne	użytkownik ma gogle na głowie i ograniczoną percepcję otoczenia	tryb stojąco/siedzący, brak chodzenia, clear play area
Overtrust	użytkownik może traktować system jako pełne szkolenie ratunkowe	jasny komunikat, że to prototyp treningowy, nie certyfikacja
Reidentyfikacja	połączenie logów z innymi danymi może ujawnić osobę	oddzielenie danych badawczych od kontaktowych, szyfrowanie
Wpływ emocjonalny	scenariusz może uruchomić lęk lub wspomnienia	kryteria wykluczenia, debriefing, możliwość odmowy udziału

Źródło: opracowanie własne

19. Wykorzystanie AI

W projekcie wykorzystano narzędzia oparte na sztucznej inteligencji, jednak ich rola miała charakter wspierający, a nie zastępujący pracę zespołu. Szczególnie istotne było użycie Claude Code przy implementacji oraz poprawianiu fragmentów mechaniki gry. Narzędzie to wspierało pisanie kodu, rozwiązywanie problemów technicznych i usprawnianie wybranych funkcji, jednak każda zmiana była następnie testowana przez zespół na fizycznych goglach Meta Quest 3/3S. Ostateczne decyzje dotyczące działania mechanik, ich poprawności oraz użyteczności były podejmowane przez członków zespołu na podstawie testów, debugowania i obserwacji działania aplikacji.

ChatGPT został wykorzystany przede wszystkim do uporządkowania struktury raportu oraz redakcji tekstu sprawozdania. AI pełniło funkcję pomocniczą w organizacji i opracowaniu dokumentacji, ale nie było źródłem niezwyfikowanych informacji o projekcie.

W module rozmowy z numerem 112 zastosowano Wit.ai (narzędzie do rozpoznawania wypowiedzi użytkownika). System umożliwiał identyfikowanie wybranych formułek i słów kluczowych wypowiedzianych przez gracza w sytuacji kryzysowej. Nasza kontrola polegała na samodzielnym dobraniu treści komunikatów, sprawdzeniu ich zrozumiałości oraz testowaniu, czy system poprawnie rozpoznaje najważniejsze elementy zgłoszenia. W razie problemów z rozpoznawaniem mowy nasz zespół modyfikował formułki i słowa kluczowe tak, aby lepiej pasowały do scenariusza.

Dodatkowo wykorzystaliśmy także syntezę mowy TTS Microsoft Paulina do wygenerowania głosu dyspozytora numeru 112. Pozwoliło to stworzyć bardziej realistyczny dialog głosowy w trakcie ćwiczenia MOWA. Wygenerowane kwestie zostały sprawdzone pod kątem zrozumiałości, naturalności oraz dopasowania do przebiegu scenariusza.

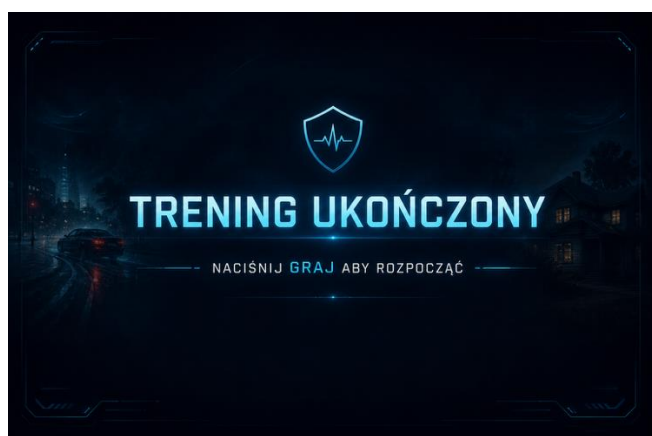
Narzędzia AI i automatyzacji zostały użyte jako wsparcie procesu projektowego, technicznego i redakcyjnego, natomiast odpowiedzialność za decyzje projektowe, testowanie oraz końcową jakość rozwiązania pozostała po stronie naszego zespołu.

20. Screeny, diagramy i dokumentacja działania

Ekran startowy – pokazuje punkt wejścia do aplikacji i wybór treningu



Trening ukończony – pokazuje przejście z treningowego trybu do scenariusza głównego



Ćwiczenie kierownica – dokumentuje mikrokrok motoryczny



Ćwiczenie mowa – dokumentuje komunikację alarmową z numerem 112



Ćwiczenie cisza – dokumentuje moduł samoregulacji



Struktura projektu – zamieszczona na Github'ie

```
CRISISREADY/  
├── Assets/  
│   ├── _Project/  
│   │   ├── Audio/           # dźwięki i nagrania (m.in. 112, TTS)  
│   │   ├── Materials/       # materiały URP  
│   │   ├── Models/          # modele 3D (auto, sypialnia, łóżko...)  
│   │   ├── Prefabs/         # prefaby sceny i UI  
│   │   ├── Scenes/          # bazowa scena (bakowana z kodu)  
│   │   ├── ScriptableObjects/ # konfiguracje aktów i scenariuszy  
│   │   ├── Textures/  
│   │   └── Scripts/  
│   │       ├── Core/        # zarządzanie sesją, stanem, audio, MR  
│   │       ├── Acts/         # logika aktów (Act1 / Act2 / Act3)  
│   │       ├── Gameplay/     # ScenarioRunner + detektory  
│   │       ├── XR/           # hand tracking, passthrough, pozy dżoni  
│   │       ├── UI/           # menu, HUD, podpowiedzi  
│   │       ├── Research/     # kwestionariusze (NASA-TLX/SUS/IPQ/UEQ)  
│   │       ├── Logging/      # zapis metrics.jsonl  
│   │       └── Editor/       # budowniki sceny i APK (LabRoomBuilder, BuildScript...)  
├── Builds/  
│   └── MRCrisisTrainer.apk  # gotowy build na Quest  
├── Packages/  
├── ProjectSettings/  
├── docs/                   # dokumentacja projektu  
├── media/  
│   └── gifs/               # materiały do README
```


21. Niepowodzenia, ograniczenia i wnioski z błędów

Podczas realizacji projektu nasz zespół napotkał szereg problemów technicznych i projektowych, które wpłynęły na sposób dalszego rozwijania aplikacji. Jednym z pierwszych wyzwań była sama instalacja gry na goglach Meta Quest. Proces ten okazał się bardziej złożony niż standardowe uruchomienie aplikacji na komputerze, ponieważ wymagał założenia konta deweloperskiego Meta, włączenia trybu developer, autoryzacji gogli przez USB oraz ręcznego sideloadingu pliku APK. W praktyce oznaczało to, że każda większa zmiana w projekcie musiała zostać zbudowana, wgrana na urządzenie i ponownie przetestowana. W kolejnych etapach pracy należałoby więc już od początku planować dodatkowy czas na buildy, instalację oraz testy sprzętowe.

Pierwotny scenariusz gry zakładał trzy sytuacje kryzysowe: udzielenie pierwszej pomocy, wyprowadzenie pojazdu z poślizgu, wezwanie pomocy i zachowanie ciszy w ukryciu przed napastnikiem. Implementacja pierwszej minigry okazała się być utrudniona z wielu względów. Pierwsze ograniczenie wiązało się z przeprowadzeniem resuscytacji przy użyciu kontrolerów. Poprawne przeprowadzenie resuscytacji wiąże się m. in. z poprawnym ułożeniem dłoni, co zostaje uniemożliwione przez kontrolery, dlatego postanowiliśmy zrezygnować z ich użycia. Jednak samo śledzenie dłoni przez program, bez kontrolerów również okazało się niemałym wyzwaniem. Nawet, gdy śledzenie było możliwe, było ono na tyle nieprecyzyjne, że sytuacja udzielania pierwszej pomocy była bardzo nierealistyczna. To, co również wpływało na brak poczucia realizmu w tej scenie to fakt, że użytkownik przeprowadzał resuscytację w powietrzu, więc nie napotykał oporu. To dodatkowo zniekształcało odczuwanie rzeczywistości i zaburzało efekt naszej aplikacji. Jako że celem naszej gry było, aby użytkownik nie potrzebował nic oprócz gogli VR, zrezygnowaliśmy z wykorzystania dodatkowych rekwizytów w świecie rzeczywistym, które mogłyby imitować reanimowanego człowieka (np. fantomy poduszka). Końcowo, problemy ze skutecznym zaplanowaniem, implementacją oraz osiągnięciem oczekiwanego efektu spowodowały, że postanowiliśmy zrezygnować całkowicie z tej sceny – dlatego końcowa forma gry nie wykorzystuje sytuacji przeprowadzania RKO.

Istotnym problemem było sterowanie wyłącznie za pomocą dłoni. Z jednej strony hand tracking zwiększał naturalność interakcji i pozwalał zrezygnować z kontrolerów, ale z drugiej strony okazał się mniej stabilny niż klasyczne sterowanie. Śledzenie dłoni bywało zawodne szczególnie przy słabszym oświetleniu, nietypowej pozycji rąk lub zasłonięciu dłoni przez inne obiekty. Zespół musiał więc wielokrotnie testować aplikację w różnych warunkach i dostosowywać mechaniki tak, aby były bardziej odporne na chwilową utratę trackingu.

Dużo pracy wymagało również poprawne ustawienie modeli 3D w scenie. Część assetów po zaimportowaniu miała niewłaściwą skalę, orientację lub punkt zaczepienia, przez co obiekty zapadały się w podłogę, lewitowały albo były obrócone w niewłaściwą stronę. Dotyczyło to między innymi elementów pokoju, łóżka, ciężarówki, telefonu czy postaci intruza. Każdy model musiał być ręcznie sprawdzany i dostosowywany do sceny, co wydłużało proces tworzenia. Z perspektywy dalszego rozwoju projektu ważne jest, aby przed włączeniem nowego assetu do gry sprawdzać jego pivot, skalę, kierunek ustawienia oraz zgodność z jednostkami używanymi w Unity.

Następnym wyzwaniem była wydajność aplikacji na goglach Quest 3/3S, ponieważ urządzenie działa samodzielnie i ma ograniczoną moc obliczeniową w porównaniu z komputerem, utrzymanie płynności rozgrywki wymagało optymalizacji sceny. Szczególnie obciążające były modele 3D, tekstury, cienie, efekty dźwiękowe oraz funkcja passthrough, która łączy obraz rzeczywistego pomieszczenia z elementami wirtualnymi. Aby uniknąć spadków liczby klatek na sekundę, konieczne było ograniczanie złożoności assetów, redukcja efektów wizualnych oraz testowanie działania aplikacji bezpośrednio na urządzeniu, a nie tylko w edytorze Unity.

Problemem okazała się także obsługa mikrofonu i rozpoznawania głosu. Moduł rozmowy z numerem 112 wymagał dostępu do mikrofonu, odpowiednich uprawnień runtime oraz dobrej jakości sygnału dźwiękowego. W praktyce rozpoznawanie wypowiedzi użytkownika mogło być zakłócanie przez hałas, niepewną wymowę, zbyt ciche mówienie albo problemy techniczne z dostępem do mikrofonu. Nasz zespół musiał więc dobrać takie formułki i słowa kluczowe, które były możliwe do rozpoznania przez system.

Osobnym wyzwaniem była naturalność dialogu z dyspozytorem numeru 112. Początkowo rozmowa brzmiała miejscami sztucznie lub zbyt mechanicznie, co mogło osłabiać realizm sytuacji. Nasz zespół musiał więc uprościć kwestie, dobrać je tak, aby pasowały do scenariusza oraz sprawdzić, czy są zrozumiałe dla użytkownika.

Istotnym ograniczeniem projektowym był również tryb siedzący Mixed Reality. Scenariusz zakładał między innymi ukrycie się pod łóżkiem, jednak użytkownik nie mógł realnie chodzić po pokoju ani fizycznie wejść pod wirtualne łóżko. Zespół musiał więc znaleźć rozwiązanie zastępcze i powiązać ukrycie z pozycją głowy użytkownika. Dzięki temu zachowano sens scenariusza, ale dostosowano go do ograniczeń bezpieczeństwa, przestrzeni oraz trybu siedzącego.

Trudności pojawiły się także w obszarze interfejsu użytkownika. Klikanie elementów menu samą dłonią było mniej dokładne niż używanie kontrolera, dlatego małe przyciski lub zbyt precyzyjne interakcje okazywały się problematyczne. W odpowiedzi na to należało zwiększać elementy interfejsu, stosować promień wychodzący z palca oraz dodawać większą tolerancję błędu przy wyborze opcji.

Ostatnim problemem technicznym był ekran końcowy, który w niektórych sytuacjach prześwitywał przez geometrię sceny lub nie wyświetlał się w pełni poprawnie. Było to związane z kolejnością renderowania oraz sposobem osadzenia elementów UI w przestrzeni świata. Problem ten pokazał, że w aplikacjach VR i MR ekranów końcowych oraz komunikatów nie należy traktować tak samo jak klasycznego interfejsu 2D.

Podsumowując, większość napotkanych problemów wynikała z pracy na rzeczywistym sprzęcie VR/MR, a nie tylko z samego projektowania aplikacji w Unity. Najważniejszą lekcją z procesu tworzenia *CrisisReady* jest to, że projekty Mixed Reality wymagają ciągłego testowania na urządzeniu docelowym, uwzględnienia ograniczeń trackingu, wydajności i przestrzeni użytkownika oraz projektowania interakcji od początku z myślą o konkretnym sposobie sterowania.

22. Upowszechnienie projektu

Projekt został udostępniony za pośrednictwem repozytorium GitHub¹. Repozytorium zawiera kod źródłowy, gotowy build APK, instrukcję instalacji, strukturę projektu, zrzuty ekranu, prezentację oraz wersję sprawozdania. Dzięki temu pełni funkcję nie tylko archiwum kodu, ale także dokumentacji technicznej umożliwiającej odtworzenie projektu przez osobę spoza zespołu.

W przyszłości wraz z rozwojem projektu przygotowalibyśmy film demonstracyjny prezentujący działanie aplikacji VR, opublikowany w serwisie YouTube. Gotowa wersja programu zostałaby udostępniona do pobrania poprzez Google Drive, dzięki czemu byłaby łatwo dostępna dla wszystkich użytkowników, niezależnie od znajomości platformy GitHub.

Dokumentacja obejmuje opis projektu, instrukcję instalacji, uruchomienia oraz wymagania sprzętowe dotyczące gogli VR.

¹ Link do repozytorium: https://github.com/JaySea-ML/CRISISREADY?fbclid=IwY2xjawSbpB9leHRuA2FlbQIxMABicmlkETAxahpFSEwyYm1MejRtMmNac3J0YwZhcHBfaWQQMjlyMDM5MTc4ODIwMDg5MgABHh-xjIYpcyOr_baPZPvN4MMvm4oE57eU4JOZZirJS6ktomjvO7CEd4yuFwKG_aem_qjHQMwZOPVQ364GevlIdUw

23. Dalszy rozwój systemu

CrisisReady może być rozwijane w kilku wzajemnie uzupełniających się kierunkach: technicznym, badawczym, użytkowym, scenariuszowym oraz edukacyjnym.

Istotnym kierunkiem rozwoju byłoby rozbudowanie elementów opartych na sztucznej inteligencji. System mógłby w przyszłości nie tylko rozpoznawać konkretne słowa kluczowe, ale także lepiej interpretować intencję wypowiedzi użytkownika, na przykład oceniać, czy zgłoszenie alarmowe zawiera najważniejsze informacje: lokalizację, opis zdarzenia, liczbę poszkodowanych i rodzaj zagrożenia. Możliwe byłoby także wprowadzenie adaptacyjnych podpowiedzi oraz personalizacji poziomu trudności. Aplikacja mogłaby dostosowywać presję czasu, liczbę wskazówek lub intensywność bodźców stresowych do doświadczenia i postępów użytkownika.

Z perspektywy badawczej kolejnym etapem powinna być pełna walidacja projektu na większej grupie uczestników. Dodatkowo można byłoby porównać grupę korzystającą z **CrisisReady** z grupą kontrolną, która otrzymuje tradycyjną instrukcję tekstową lub ogląda film szkoleniowy. Szczególnie ważnym elementem byłby także test transferu, czyli sprawdzenie czy umiejętności wyćwiczone w środowisku MR/VR przenoszą się na bardziej realistyczne sytuacje poza aplikacją.

Projekt można byłoby również rozwijać pod kątem UX. W dalszych wersjach warto poprawić czytelność interfejsu, powiększyć przyciski, uprościć sposób interakcji dla osób bez doświadczenia z VR oraz dodać krótką kalibrację użytkownika przed rozpoczęciem scenariusza. Taki tryb wprowadzający mógłby wyjaśniać, jak działa śledzenie dłoni, w jaki sposób wybiera się opcje w menu i jak należy reagować na komunikaty w grze. Dzięki temu aplikacja byłaby bardziej dostępna dla dzieci, osób starszych lub użytkowników, którzy po raz pierwszy korzystają z gogli VR.

Największy potencjał rozwojowy dotyczy jednak rozbudowy scenariuszy. **CrisisReady** można byłoby rozszerzyć o kolejne sytuacje kryzysowe, takie jak udzielanie pierwszej pomocy, pożar, ewakuacja z budynku, zagubienie się dziecka, załabnięcie osoby w miejscu publicznym, agresywne zachowanie obcej osoby czy różne warianty wypadków drogowych. Obecny scenariusz poślizgu samochodu mógłby zostać rozwinięty w moduł przeznaczony dla szkół jazdy, gdzie kursanci mogliby bezpiecznie trenować reakcje na utratę kontroli nad pojazdem, nagłe wtargnięcie przeszkody na drogę albo konieczność wykonania manewru awaryjnego. Z kolei moduły dotyczące rozmowy z numerem alarmowym i zachowania ciszy mogłyby zostać

zaadaptowane jako narzędzie edukacyjne dla szkół, uczące dzieci i młodzież, jak zachowywać się w sytuacjach zagrożenia, jak rozmawiać z dyspozytorem oraz jak opanować stres.

W dłuższej perspektywie *CrisisReady* mogłoby stać się szerszą platformą treningową do nauki reakcji kryzysowych. Takie narzędzie mogłoby być wykorzystywane nie tylko w edukacji szkolnej, ale także w szkołach jazdy, na kursach pierwszej pomocy, w szkoleniach BHP, warsztatach bezpieczeństwa czy programach profilaktycznych. Zaletą takiego rozwiązania jest możliwość wielokrotnego przećwiczenia trudnych sytuacji w bezpiecznym środowisku, bez realnego narażania użytkownika na niebezpieczeństwo. Mixed Reality pozwala jednocześnie zachować element immersji i stresu sytuacyjnego, który jest trudny do osiągnięcia w tradycyjnych formach szkolenia.

Równolegle należałoby rozwijać kwestie etyczne i bezpieczeństwa danych. W przyszłych wersjach aplikacji konieczne byłoby opracowanie pełnego protokołu świadomej zgody, szczególnie jeśli system byłby testowany na dzieciach lub osobach niepełnoletnich. Należałoby również jasno informować użytkowników, jakie dane są zbierane, w jakim celu są analizowane i jak długo będą przechowywane. Preferowanym rozwiązaniem byłoby lokalne przetwarzanie danych, ograniczenie zapisu nagrań głosu oraz umożliwienie użytkownikowi kontroli nad ustawieniami prywatności. Dzięki temu rozwój *CrisisReady* mógłby łączyć innowacyjność technologiczną z odpowiedzialnym podejściem do edukacji, bezpieczeństwa i ochrony danych.

24. Wnioski końcowe

CrisisReady pokazuje, że możliwe jest stworzenie prototypu technologii kognitywnej w MR/VR, która nie ogranicza się do prezentowania sceny, lecz aktywnie wspiera użytkownika w uczeniu się reakcji kryzysowych. Projekt integruje trzy typy działania: motoryczne, komunikacyjne i samoregulacyjne. Każdy z nich został powiązany z mikrokrokami, możliwością feedbacku i metrykami do przyszłej walidacji.

Największą wartością projektu jest połączenie narracyjnego scenariusza z logiką mikroprogresji. Użytkownik nie wykonuje izolowanych ćwiczeń, ale przechodzi przez spójną sytuację kryzysową, w której wcześniejsze zdarzenia nadają sens późniejszym decyzjom. Jednocześnie system zachowuje strukturę badawczą: poszczególne kroki można mierzyć, porównywać i analizować.

Najważniejsze ograniczenia dotyczą stabilności technicznej, naturalności dialogu, warunków działania hand trackingu oraz braku pełnego badania użytkowników. Nie obniża to jednak wartości prototypu, ponieważ projekt osiągnął etap grywalnej demonstracji i zawiera fundamenty do dalszej walidacji.

Bibliografia

Literatura:

- Brooke, J. (1986). SUS: A “quick and dirty” usability scale. W: P. W. Jordan, B. Thomas, B. A. Weerdmeester, & A. L. McClelland (red.), Usability Evaluation in Industry, 189–194.
- Hart, S. and Staveland, L. (1988) Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In: Hancock, P. and Meshkati, N., Eds., Human Mental Workload, North Holland, Amsterdam, 139-183.
- Kennedy, R. S., Lane, N. E., Berbaum, K. S., & Lilienthal, M. G. (1993). Simulator sickness questionnaire: An enhanced method for quantifying simulator sickness. The international journal of aviation psychology, 3(3), 203-220.
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. Cognitive Science, 12(2), 257–285.
- Wang, C. W., Lu, P. C., Cheng, Y. C., & Chen, M. Y. (2025, April). MR. Drum: Designing Mixed Reality Interfaces to Support Structured Learning Micro-Progression in Drumming. In Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, 1-17.
- Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. Journal of child psychology and psychiatry, 17(2), 89-100.

Dokumentacja techniczna i źródła narzędzi

- Unity Technologies. (2026). Unity 6 User Manual. Unity Documentation.
- Khronos Group. (n.d.). OpenXR Specification. Khronos Registry.
- Meta. (n.d.). Meta XR SDKs for Unity. Meta Horizon OS Developers Documentation.
- Meta. (n.d.). Wit.ai documentation. Wit.ai.
- Microsoft. (n.d.). Text to speech overview – Azure AI Speech. Microsoft Learn.
- Microsoft. (n.d.). Language and voice support for Azure AI Speech. Microsoft Learn.
- JASP Team. (n.d.). JASP: A fresh way to do statistics. JASP.

Źródła assetów i zasobów multimedialnych

- Adobe. (n.d.). Mixamo.
- Sketchfab. (n.d.). Sketchfab 3D models platform.
- Itch.io. (n.d.). Game assets and resources.
- Freesound. (n.d.). Freesound: Collaborative database of Creative Commons licensed sounds.

Spis tabel

Tabela 1. Problem projektowy i odpowiedź systemu CrisisReady	4
Tabela 2. Pytania badawcze i proponowane wskaźniki walidacji.....	7
Tabela 3. Kryteria sukcesu prototypu CrisisReady	7
Tabela 4. Fazy scenariusza, tryb środowiska i funkcje poznawcze	8
Tabela 5. Mikro kroki, dane wejściowe i przykładowe kryteria zaliczenia	9
Tabela 6. Realizacja pętli poznawczej.....	11
Tabela 7. Zasady UX/UI i ich zastosowanie w projektowaniu interakcji kryzysowej.....	12
Tabela 8. Wykorzystane technologie, narzędzia i zasoby projektu	16
Tabela 9. Typy funkcji realizowanych przez program	17
Tabela 10. Zmienne zapisywane w logach metrics.jsonl i ich znaczenie badawcze.....	18
Tabela 11. Harmonogram etapów pracy nad projektem.....	19
Tabela 12. Zakres testowania technicznego prototypu.....	23
Tabela 13. Proponowana procedura badania użytkowników i dane zbierane na kolejnych etapach	24
Tabela 14. Miary efektywności, UX i obciążenia poznawczego	25
Tabela 15. Ryzyka etyczne, prywatnościowe i bezpieczeństwa oraz sposoby ich minimalizacji	29